



REFERENCE MANUAL

Feature atlas, legacy shell, testing, plugin examples

Open charting PWA — own the axes, swap the engine



CONTENTS

01	Feature atlas
02	Legacy shell
03	Testing reference
04	Plugin examples



FEATURE ATLAS

Map of AXIS product surfaces to repository paths: plugins, AXIS, worker, storage, test, legacy.

FEATURE ATLAS

Abstract

This atlas is a **path index** for AXIS. Use it to jump from a product feature to the code that implements it. Prefer Solid product paths over legacy static shell files.

Conceptual model

```
mindmap
  root((AXIS))
    Plugins
      sources
      streams
      engines
      storage
      loader
    AXIS
      chart
      editor
      ui
      store
    Worker
      run
      keys
      scripts
      DO
    DevOps
      vite
      pages
      ci
```



Atlas tables

Plugin system

Feature	Primary paths
Contracts	<code>frontend/src/plugins/types.ts</code>
Registry	<code>frontend/src/plugins/registry.ts</code>
Bootstrap	<code>frontend/src/plugins/bootstrap.ts</code>
Active set	<code>frontend/src/plugins/active.ts</code>
Dynamic install	<code>frontend/src/plugins/loader.ts</code>
Barrel	<code>frontend/src/plugins/index.ts</code>
Manager UI	<code>frontend/src/ui/PluginManager.tsx</code> , <code>plugin-badges.tsx</code>
Docs (in-tree)	<code>frontend/src/plugins/README.md</code>

Data plugins

Feature	Primary paths
Sources	<code>frontend/src/sources/catalog.ts</code> , <code>upload-store.ts</code>
Streams	<code>frontend/src/streams/catalog.ts</code> , <code>multiplex.ts</code>
Engines	<code>frontend/src/engines/catalog.ts</code>
Load symbol	<code>frontend/src/data/load-symbol.ts</code> , <code>parse-bars.ts</code>
Indicator run	<code>frontend/src/indicators/runner.ts</code>

Storage / library

Feature	Primary paths
Catalog	<code>frontend/src/storage/catalog.ts</code>
Local IDB	<code>frontend/src/storage/local.ts</code> , <code>idb.ts</code>
Cloud	<code>frontend/src/storage/cloud.ts</code>
Git	<code>frontend/src/storage/git.ts</code> , <code>git-github.ts</code> , <code>git-gitlab.ts</code>
Service façade	<code>frontend/src/storage/service.ts</code>
Library panel	<code>frontend/src/ui/ScriptLibraryPanel.tsx</code>



UI (UI)

Feature	Primary paths
Entry	<code>frontend/src/index.tsx</code> , <code>app.tsx</code>
Chart host	<code>frontend/src/chart/ChartHost.tsx</code> , <code>manager-access.ts</code> , <code>pane-manager.ts</code> , <code>series-factory.ts</code>
Drawings	<code>frontend/src/chart/drawing-layer.ts</code> , <code>pine-drawings.ts</code>
Editor	<code>frontend/src/editor/*</code>
Results	<code>frontend/src/results/*</code> , <code>ui/ResultsPanel.tsx</code>
Store	<code>frontend/src/store/index.ts</code> , <code>types.ts</code>
Topbar / settings	<code>frontend/src/ui/Topbar.tsx</code> , <code>SettingsDialog.tsx</code>

Worker

Feature	Primary paths
Router	<code>frontend/worker/src/index.ts</code>
Run	<code>frontend/worker/src/runtime.ts</code> , <code>pyodide_runtime.ts</code>
Auth / keys	<code>frontend/worker/src/auth.ts</code> , <code>keys.ts</code>
Scripts	<code>frontend/worker/src/scripts.ts</code> , <code>schemas/scripts.sql</code>
Session DO	<code>frontend/worker/src/durable-objects/session.ts</code>
Config	<code>frontend/worker/wrangler.toml</code> , <code>RUNTIME.md</code> , <code>README.md</code>

Build / ops

Feature	Primary paths
Package scripts	<code>frontend/package.json</code> (<code>test:*</code> , <code>build</code> , <code>dev</code>)
Static server	<code>frontend/axis_pwa_server.py</code>
Public assets	<code>frontend/public/plugins</code> , <code>public/vendor</code> , <code>public/pyodide</code>
Makefile	<code>run-axis</code> (Vite), <code>pages-deploy</code> → <code>dist/</code> , <code>test-frontend</code>
CI	<code>.github/workflows/ci.yml</code> (coverage + e2e smoke), <code>axis-nightly.yml</code>
Coverage gate	<code>frontend/scripts/check-coverage.mjs</code> (scoped core ≥95%)



Examples & tests

Feature	Primary paths
Example plugins	<code>frontend/public/plugins/*.js</code> , <code>src/plugins/example-*.js</code>
Unit tests	<code>frontend/tests/**</code>
Worker tests	<code>frontend/worker/tests/**</code>
E2E	<code>frontend/e2e/**</code> , <code>playwright.config.ts</code>
Testing guide	<code>frontend/TESTING.md</code>
Legacy notes	<code>frontend/LEGACY.md</code>

Namespaces & keys

Key / namespace	Use
<code>pynescrypt.axis.plugins.v1</code>	Installed dynamic plugins
<code>pynescrypt.axis.library.v1</code>	Local library LS fallback
<code>pynescrypt.axis.storage</code>	IDB database name
<code>pynescrypt.axis.v1</code>	App state (target; migrates from superchart)
<code>pynescrypt.superchart.*</code>	Legacy keys still read for migration
Contract ns	<code>pynescrypt.axis.plugins.v1</code> (conceptual)

Infrastructure constants

Name	Value
CF project	<code>pynescrypt-superchart</code>
Health service	<code>pynescrypt-axis-worker</code>
Pyodide version	<code>0.26.2</code> (self-hosted path)

See also

- [Plugins](#)
- [Worker](#)
- [Legacy shell](#)



LEGACY SHELL

Pre-Solid SuperChart static shell vs current AXIS product path — what to ignore and what still runs.

LEGACY SHELL

Abstract

AXIS’s shipping product path is **Solid + Vite**. An older **static shell** remains in the repo for historical scripts and some Bun tests. Do not document or extend the legacy shell as the primary UI.

Source of truth: `frontend/LEGACY.md`.

Conceptual model

```
flowchart LR
    LEG[Legacy main.js tree]
    CUR[Solid index.tsx]
    LEG -->|keep for tests| BUN[bun test static bits]
    CUR --> SHIP[dist / Pages]
```

What is legacy

Path	Role
<code>frontend/src/main.js</code>	Old bootstrap (chart.js, topbar.js, ...)
<code>frontend/style.css</code>	TV-blue-era tokens
<code>frontend/pine-editor.js</code>	Pre-Solid CM6 wiring
<code>frontend/server.ts</code>	Bun static server for old tree
Root / frontend <code>sw.js</code> (static shell SW)	Service worker for old shell

Makefile `make run-frontend` still prints SuperChart Lite and runs `bun run frontend/server.ts` on `:8081` — useful for legacy debugging, **not** the Solid product path.

What is current

Path	Role
<code>frontend/src/index.tsx</code> , <code>app.tsx</code>	Solid app
<code>frontend/src/chart/ChartHost.tsx</code>	LWC panes
<code>frontend/src/ui/*</code>	Topbar, Settings, Results, Manager, ...
<code>frontend/public/</code> + <code>bun run build</code>	Production PWA assets
<code>frontend/axis_pwa_server.py</code>	Serve dist for demos



Migration residue

Residue	Handling
<code>pynescrypt.superchart.plugins.v1</code>	Loader still reads
<code>pynescrypt.superchart.library.v1</code>	Local storage migrates
<code>pynescrypt.superchart.editor.doc</code>	Draft migration
CF project <code>pynescrypt-superchart</code>	Frozen name despite product rebrand
Some docs/Makefile strings	Still say SuperChart Lite

App state target namespace: `pynescrypt.axis.v1` (migrates from `pynescrypt.superchart.*`).

Invariants

1. **Do not port new features into `main.js`.**
2. **Prefer Solid store** (`src/store`) over `state.js`.
3. **Plugin registry** is the Solid path (`src/plugins/registry.ts`); dual registries in old JS should be treated as dead ends.
4. **Pages deploy** should use Vite `dist/`, not the repo root static tree.

Failure modes

Confusion	Clarification
“UI doesn’t match docs”	You may be serving the legacy shell
Plugin manager missing	Legacy topbar lacks Solid Manager
Tests pass, UI wrong	Unit tests may still import legacy helpers

See also

- [Feature atlas](#)
- [Build and serve](#)
- [AXIS hub](#)



TESTING REFERENCE

AXIS unit, worker, security, coverage gate, and Playwright e2e conventions with paths and commands.

TESTING REFERENCE

Abstract

AXIS tests are **Bun-first** under `frontend/`, with Playwright for browser smoke and a scoped coverage ratchet. Canonical human guide: `frontend/TESTING.md`. This page is the reference map for agents and contributors.

Conceptual model

```
flowchart TB
    U[Unit Bun] --> CORE[plugins storage store catalogs]
    W[Worker Bun] --> API[auth keys scripts runtime]
    S[Security] --> THREAT[URL schemes partitions If-Match]
    E[Playwright] --> SMOKE[@smoke PR]
    E --> FULL[nightly all]
    C[Coverage gate] --> SCOPED[95% scoped lines]
```

Commands

```
cd frontend

bun run test           # unit + worker tests
bun run test:unit      # frontend/tests only
bun run test:coverage  # lcov + text → coverage/
bun run test:coverage:gate # coverage + 95% scoped gate
bun run test:security  # tests/security/
bun run test:e2e:smoke  # Playwright @smoke
bun run test:e2e:critical # @smoke|@critical
bun run test:e2e       # all
bun run test:all       # gate + security
```

Root:

```
bun run test:frontend
bun run test:frontend:coverage
make test-frontend
```



Layout

```
frontend/tests/
  setup.ts
  fixtures/
  helpers/
  *.test.ts
  integration/
  security/
frontend/scripts/check-coverage.mjs
frontend/e2e/
  smoke.spec.ts
frontend/playwright.config.ts
frontend/worker/tests/
  auth.test.ts
  keys-*.test.ts
  scripts*.test.ts
...
```

Coverage policy

Item	Policy
Gate	95% lines on scoped core
Ratchet history	70 → 80 → 83 → 87 → 90 → 95
Include	plugins, storage (minus idb), store, results, sources, streams, data, chart pure helpers + series-factory/manager-access, worker auth/keys/runtime/scripts
Exclude from gate	drawing-layer.ts , pane-manager (unit elsewhere), runner chart apply, pyodide boot, legacy JS, Solid UI .tsx
Full report	bun test --coverage unscoped

Conventions

1. `import './setup'` first when tests touch store/plugins/storage.
2. **No real network** — mock `fetch` / `WebSocket` .
3. Table-driven tests for catalog built-ins.
4. Plugin fixtures in `tests/fixtures/plugins/` .
5. Prefer calling handlers with fake params over full browser for unit scope.

Playwright

Config (`playwright.config.ts`):

- `testDir: ./e2e`
- Chromium project
- `webServer: bun run build && bun run preview -- --host 127.0.0.1 --port 4173`
- `baseUrl: http://127.0.0.1:4173` (override `AXIS_E2E_BASE`)
- Timeout 60s; CI retries 1



Smoke mocks `/run` and `Binance`. Use `data-testid` attributes (`axis-btn-load`, `axis-manager`, ...).

Security suite

`bun run test:security` covers:

- Plugin URL scheme rejection
- Storage-via-URL rejection
- Poisoned `localStorage`
- Worker partition isolation
- If-Match 409
- Admin keys

Adding a unit test

1. Create `frontend/tests/<area>.test.ts`.
2. Import setup if needed.
3. Use fixtures under `tests/fixtures/`.
4. Run `bun run test:unit` and `bun run test:coverage:gate`.

CI linkage

See [CI and testing](#) for workflow job names and nightly schedule.

Failure modes

Failure	Mitigation
Coverage drop	Add tests in scoped files or deliberately exclude only with policy change
E2E flake	Mock network; single worker; stable testids
IDB in unit	Local storage falls back to memory maps

See also

- [CI and testing](#)
- [Plugin contracts](#)
- [Worker auth](#)



PLUGIN EXAMPLES

Worked AXIS example plugins: CoinGecko source, Tiny Pine engine, Cloudflare DO stream — load paths and contracts.

PLUGIN EXAMPLES

Abstract

AXIS ships three **loadable example plugins** as ES modules under `frontend/public/plugins/` (also mirrored for reference under `frontend/src/plugins/example-*.js`). They are not built-ins; install via Manager → Plugins → **Load from URL**.

Full narrative: `frontend/src/plugins/README.md`.

Conceptual model

```
flowchart LR
    URL[""/plugins/example-*.js"]
    URL --> LOADER[loadPluginFromUrl]
    LOADER --> REG[registry]
    REG --> PICKER[Source/Stream/Engine pickers]
```

Loading

1. Serve the PWA (Vite dev, preview, or `axis_pwa_server.py` on `dist`).
2. Open **Manager** → **Plugins**.
3. Paste a same-origin URL, for example:

```
http://127.0.0.1:8081/plugins/example-coingecko-source.js
http://127.0.0.1:3000/plugins/example-tiny-pine-engine.js
http://127.0.0.1:8081/plugins/example-cf-do-stream.js
```

4. **Load** — registry updates; pickers show the new plugin.
5. Persistence: `localStorage` `pynescrypt.axis.plugins.v1` restores on next visit.

You may host modules on any origin that allows **module** CORS (same-origin is simplest).

Example catalog

File	id	kind	Purpose
<code>example-coingecko-source.js</code>	<code>coingecko</code>	source	Public CoinGecko market_chart → synthetic OHLC
<code>example-tiny-pine-engine.js</code>	<code>tiny-pine</code>	engine	In-browser JS DSL: sma/ema/rsi/plot/strategy
<code>example-cf-do-stream.js</code>	<code>cf-do</code>	stream	WS to Worker <code>/api/stream</code> SessionDO

Paths:

- Served: `frontend/public/plugins/`



- Source copies: `frontend/src/plugins/example-*.js`
-

CoinGecko source

Contract: `kind: 'source', fetchHistorical`.

Config: `baseUrl` (default CoinGecko v3), `vsCurrency` (default `usd`).

Behavior:

- Maps common symbols (`BTC` → `bitcoin`, ...).
- Requests `market_chart` for a day window derived from `interval` × bar budget.
- Synthesizes OHLC from consecutive price points (API is not full candles).

Limits: public rate limits (~10–30 calls/min); CORS depends on CoinGecko policy.

Use when: demo custom source without a key.

Tiny Pine engine

Contract: `kind: 'engine', isReady, run`.

Behavior:

- Tokenizes a tiny expression language.
- Built-ins: `close`, `open`, `high`, `low`, `volume`, `sma`, `ema`, `rsi`, `plot`, `strategy.entry` / `strategy.close`.
- Returns `RunResult` with plots/events — **not** full PYNE fidelity.

Use when: offline demos without loading ~14MB Pyodide or hitting Flask.

Cloudflare DO stream

Contract: `kind: 'stream', start` → stop.

Config: `endpoint` — Worker base URL (`https://...` or `http://127.0.0.1:8787`). Empty → error.

URL built:

```
{endpoint as wss}/api/stream?session={symbol}@{interval}&symbol=...&interval=...
```

Requires: Worker with `SESSIONS` Durable Object bound and deployed ([durable objects](#)).

Use when: many tabs share one upstream Binance kline subscription.



Minimal authoring templates

Source

```
id: 'my-source',
name: 'My Source',
kind: 'source',
description: 'Shown in Manager',
configSchema: {
  baseUrl: { type: 'string', default: 'https://example.com', label: 'Base URL' },
},
async fetchHistorical({ symbol, interval, config }) {
  const res = await fetch(`${config.baseUrl}/bars?symbol=${symbol}&interval=${interval}`);
  return res.json(); // Bar[]
},
};
```

Stream

```
id: 'my-stream',
name: 'My Stream',
kind: 'stream',
start({ symbol, interval, onBar, onError, onStatus }) {
  const ws = new WebSocket('wss://example.com/stream');
  ws.onopen = () => onStatus({ state: 'open' });
  ws.onmessage = (ev) => {
    const b = JSON.parse(ev.data);
    onBar({ time: b.t, open: b.o, high: b.h, low: b.l, close: b.c, volume: b.v });
  };
  ws.onerror = () => onError(new Error('ws error'));
  return () => ws.close();
},
};
```

Engine

```
id: 'my-engine',
name: 'My Engine',
kind: 'engine',
async isReady() { return true; },
async run({ script, bars }) {
  return { status: 'success', plots: bars.map((b) => b.close), events: [], series: {}, meta: {} };
},
};
```

Storage plugins **cannot** be loaded from URL yet.

Invariants & edge cases

1. Treat plugin URLs as **executable code**.



2. `normalizePluginUrl` rewrites `/src/plugins/` → `/plugins/` for production.
3. Dangerous schemes rejected (`javascript:` , `vbscript:` , `data:text/html`).
4. After load, window may emit product-specific events (see README) for UI extensions.

Failure modes

Issue	Fix
Failed to restore URL	404 after deploy path change
CORS on CoinGecko	Offline mock or proxy
DO stream errors	Bind SessionDO; set endpoint
Tiny engine parse errors	Unsupported DSL constructs

See also

- [Dynamic loader](#)
- [Contracts](#)
- [Worker durable objects](#)